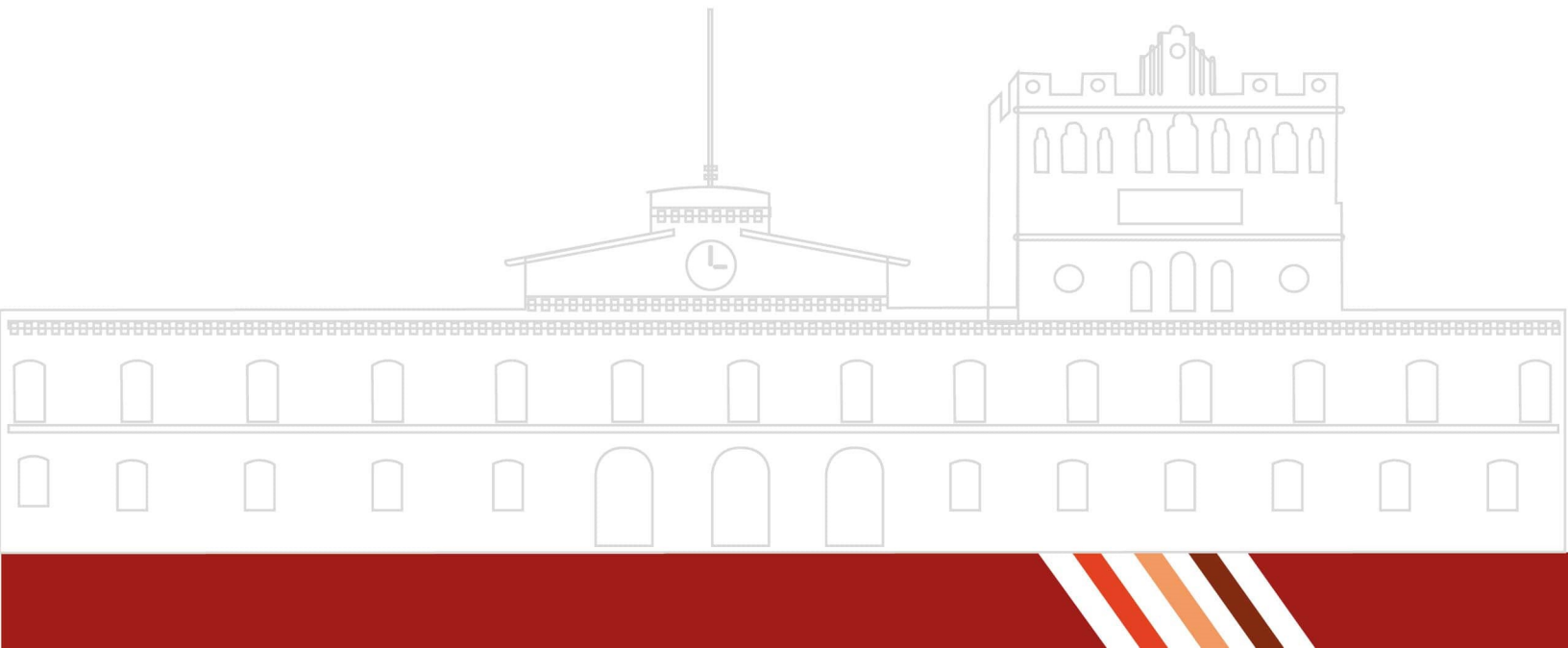


REPORTE DE PRACTICA NO. 1

Analisis de Expresiones Regulares

ALUMNO:

Andres Garcia Colmenares



1. Introduccion

Practica 1 - Analisis de Expresiones Regulares

En esta practica, se creara al final un programa para expresiones regulares, las cuales son de gran utilidad, pero principalmente se usan en compiladores, para poder identificar palabras claves, palabras reservadas, numeros, etc, y con esto mismo poder asignar cada token a una categoria, la cual el compilador se encargara de verificar que cada token este en un lugar correcto y que a su vez, pueda ser usado para una funcion especifica o por una variable

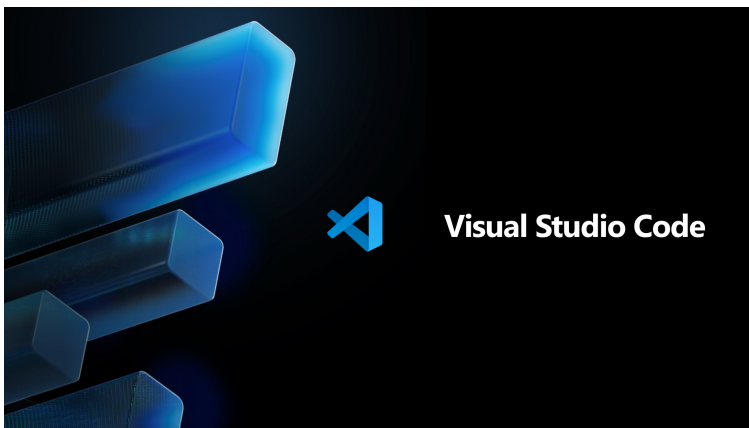
2. Herramientas empleadas

Para el desarrollo de la práctica se emplearon las siguientes herramientas:

1. Java Development Kit (JDK). Usado para compilacion y ejecucion de aplicaciones



2. Visual Studio Code. Usado para desarrollar el codigo fuente del programa.



3. Desarrollo

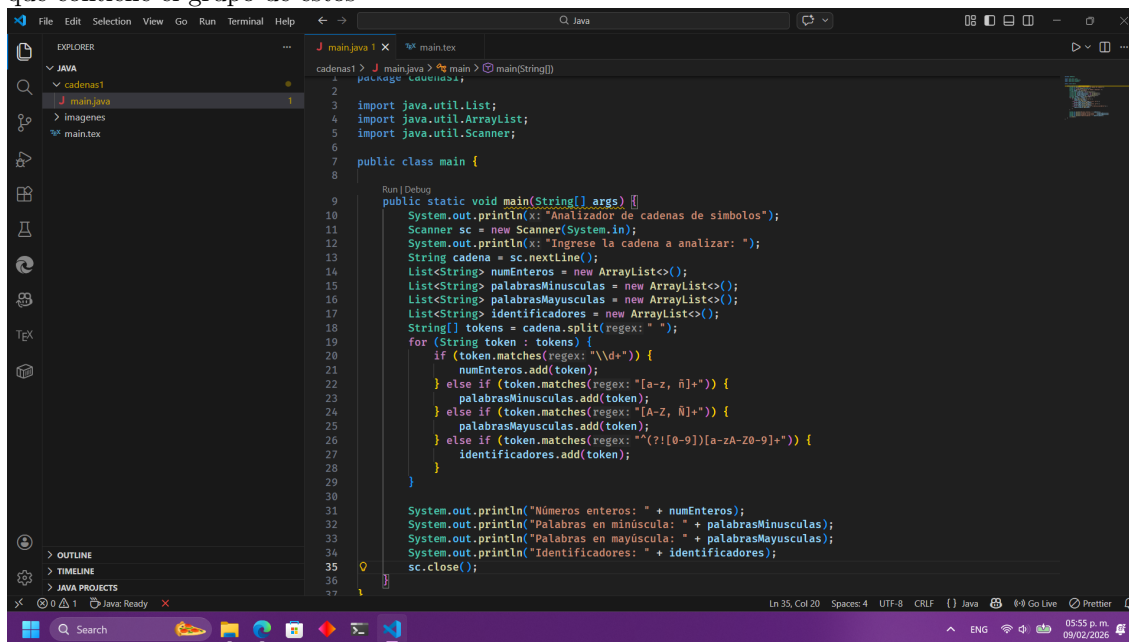
Análisis de requisitos

Para lo siguiente se requiere primero, crear una expresion regular que procese cada tipo de cadena de caracteres segun su patron, por ejemplo, un numero entero, solo requiere que tenga 1 o mas digitos, por lo tanto su expresion regular seria: `[0-9]+`, otro ejemplo, una palabra en minuscula, solo requiere que tenga 1 o más letras minusculas, por lo tanto su expresion regular seria: `[a-z]+`, sin contar la ñ para evitar problemas de momento

Expresiones regulares usadas

1. Numero entero: `[0-9]+`
2. Palabra en minuscula: `[a-z]+`
3. Palabra en mayuscula: `[A-Z]+`
4. Identificadores: `[^(?![0-9])[a-zA-Z0-9]+]`

Teniendo todo esto, se puede realizar un programa basico, el cual con una cadena finita de caracteres, se puede realizar una separacion por cada espacio con el metodo `split(" ")`, y unido con el metodo `matches()` se puede validar cada token (llamado asi por el compilador) con cada una de las expresiones regulares, para asi identificar que token corresponde a cada tipo de expresion regular, y asi asignar cada token a un arreglo que contiene el grupo de estos



```
1 package cadenas1;
2
3 import java.util.List;
4 import java.util.ArrayList;
5 import java.util.Scanner;
6
7 public class main {
8
9     public static void main(String[] args) {
10         System.out.println("Analizador de cadenas de simbolos");
11         Scanner sc = new Scanner(System.in);
12         System.out.println("Ingrese la cadena a analizar: ");
13         String cadena = sc.nextLine();
14         List<String> numEnteros = new ArrayList<>();
15         List<String> palabrasMinusculas = new ArrayList<>();
16         List<String> palabrasMayusculas = new ArrayList<>();
17         List<String> identificadores = new ArrayList<>();
18         String[] tokens = cadena.split(regex: " ");
19         for (String token : tokens) {
20             if (token.matches(regex: "\\d+")) {
21                 numEnteros.add(token);
22             } else if (token.matches(regex: "[a-z, ñ]+")) {
23                 palabrasMinusculas.add(token);
24             } else if (token.matches(regex: "[A-Z, Ñ]+")) {
25                 palabrasMayusculas.add(token);
26             } else if (token.matches(regex: "^(?![0-9])[a-zA-Z0-9]+")) {
27                 identificadores.add(token);
28             }
29         }
30
31         System.out.println("Números enteros: " + numEnteros);
32         System.out.println("Palabras en minúscula: " + palabrasMinusculas);
33         System.out.println("Palabras en mayúscula: " + palabrasMayusculas);
34         System.out.println("Identificadores: " + identificadores);
35         sc.close();
36     }
37 }
```

4. Cuestionario

¿Cuál es la utilidad de las expresiones regulares en la identificación de palabras de un lenguaje?

Son de buen uso para la creación de patrones que resulten en una identificación de datos para verificar si un dato es válido o no

¿Cuáles elementos del lenguaje de programación utilizaste para la implementación de las estructuras de datos?, ¿Cómo las utilizaste? Se usó el método `splits()` y el método `matches()` en Java, se utilizaron con el token y como una función junto con su expresión regular

¿Cuál es el proceso que seguiste para analizar las cadenas de símbolos?

1. Verificar que símbolos son, 2. Revisar que fila de caracteres se usará, 3. Crear una expresión regular simbólica con una página web, 4. Copiar y verificar la expresión con nuevos símbolos

¿Qué elementos del lenguaje de programación te permitieron implementar expresiones regulares? Las cadenas de caracteres (`string`) ya tienen varias funciones para separar y modificarlas según una expresión regular

¿Qué condiciones o recomendaciones se deben seguir al escribir las expresiones regulares? Usar correctamente los símbolos especiales, como `?` (para opcional), `+` (para una cadena infinita), etc

5. Conclusiones

Con el programa se fortaleció el conocimiento de expresiones regulares las cuales son necesarias para frontends en páginas web, aplicaciones u otro tipo de datos que necesite un formato específico, especialmente datos que puedan romper el programa

Referencias Bibliográficas

References

- [1] TecnoDigital (**2025, 10 febrero**). Expresiones regulares (RegEx): Guía completa y ejemplos. Informática y Tecnología Digital. <https://informatecdigital.com/expresiones-regulares-regex/>
- [2] Expresiones Regulares en Java: Guía Completa para su Uso. (**2025, 16 julio**). 4Geeks <https://4geeks.com/es/lesson/expresiones-regulares-java>